Factor Pose and Retirement-Invariant Checkpoint

Tactix Development Project

August 10, 2026

Executive summary

This checkpoint advances the open Phase-4 `mod4block` fixture in two ways. First, it closes two remaining Factor application-order details: Tactix now applies a selected quotient chain from the last factor backward, and emits each large quotient by preserving the chosen last row's clause pose rather than the sorted remainder key. The shipped-default event stream now matches Kissat through conflict 346. Conflict 347 is the already known policy difference where Kissat performs first-round conflict reuse and the shipped Tactix default does not.

Second, a matched-policy diagnostic with conflict reuse enabled moves the causal frontier substantially later. Decisions, conflicts, and backtracks remain identical through conflict 1009. Conflict 1010 is the first different learned-clause/backjump outcome. An earlier textual difference retains a root-false literal `-89` in a Tactix clause, but does not alter the decision/backtrack sequence until this later analysis. The next controlled increment is to trace the conflict-1007–1010 reasons back to that clause's creation or simplification boundary. Factor should be reopened only if equal input state reveals a grid, reference, update, or emission mismatch.

The same production experiment exposed an independent correctness defect: a long release run exhausted the VMTF queue. Analyze could bump a retired variable, and Substitute could rediscover a retired variable through stale binary edges and deactivate it twice. Both active-variable boundaries are now explicit and adversarially tested. A 120-second diagnostic ran without queue corruption.

This is not yet a shipped-performance close. The clean shipped-default pin does not solve within 120 seconds. Conflict reuse solves this one fixture, but previous broad evidence found it globally neutral-to-negative with severe fragile losses, so the default is unchanged.

1 Implementation decisions

1.1 Factor quotient construction

The selected Factor grid is an ordered multiset of clause occurrences, not a set of sorted remainders. Two implementation details are consequently load-bearing:

1. The selected factor chain is consumed in Kissat's `q->prev` order. Divider additions, LRAT resolution groups, grid deletions, and candidate updates all follow that reverse order.
2. A large quotient copies the selected last-factor clause's existing literal order and removes that factor. The interned sorted remainder remains only an equality key. This preserves the initial watched pair after sparse watch reconstruction.

Focused tests cover reverse application, sparse-resume binary order, repeated large occurrences, assigned remainders, and a deliberately non-sorted last-row pose. The Factor calculus and implementation ledger retain the rule as one indivisible occurrence-preserving mechanism.

1.2 Retired-variable lifecycle

The release crash was localized using three progressively narrower Slurm diagnostics. Queue exhaustion first appeared after roughly 1.39 million conflicts. A reciprocal-link alarm then found variable 1295 disconnected while it was still counted as active. Two mechanisms could violate that invariant:

- the queue and score bump helpers counted a retired-variable bump but continued into the reconnect/update operation; and
- Substitute's SCC roots and edges treated every unassigned literal as active, so stale binaries mentioning an already substituted extension could produce a second retirement.

Both bump helpers now return immediately for a retired variable. Substitute requires an unassigned, non-retired variable for SCC roots, SCC edges, and its LRAT binary-path traversal. The tests directly bump a retired variable and verify unchanged queue endpoints, links, heap score, and population; a second fixture retains stale equivalence binaries around a retired variable and requires no duplicate SCC or retirement event.

2 Correctness and performance evidence

Table 1 records the primary evidence. SAT results described as verified passed the run harness's independent model and DRAT checks.

Purpose	Artifact / job	Result and disposition
Shipped production safety	job `1607092`, production-v1 window	No crash or verdict at the 120.12 s timeout; clean trace-off pin `6a25d789...`
Conflict-reuse production diagnostic	job `1607093`, `...-conflict-reuse-v1`	Tactix SAT in 48.56 s, 791,823 conflicts, verified; Kissat SAT in 18.61 s, verified; diagnostic only
Matched-policy detailed trace	job `1607094`, `...-reuse-conflict1400-v1`	Tactix SAT in 59.82 s and Kissat SAT in 18.80 s, both verified; decision/conflict/backtrack parity through conflict 1009
Queue-corruption diagnostic	job `1607089`	Completed the full 120 s diagnostic window without reciprocal-link failure, assertion, or crash
Clean local Release suite	clean verification build	68 of 68 tests passed; only the known slow scripted-default demo was excluded
Repository hygiene	documentation and production-symbol checks	Document budgets, comment hygiene, whitespace, and zero-overhead event-symbol checks all passed

Table 1: Evidence for the retained checkpoint.

The parity-through-1009 statement is measured. The attribution of the conflict-1010 outcome to the earlier clause/reason lifetime is currently inferred: the payload difference is observable, but its owning transform has not yet been proven. It may be Factor emission, another startup simplification, or later watch/clause maintenance. This uncertainty is why no additional semantic change is included in this checkpoint.

The reuse result is also diagnostic rather than an acceptance result. The fixture improves enough to show that the conflict-347 policy boundary explains substantial work here. The existing Tier-B

report, however, measured roughly two percent aggregate loss and severe per-instance regressions from the same mechanism. A single 48.56-second solve therefore does not override the project's cumulative acceptance contract.

3 Limitations and unresolved risks

- The shipped-default `mod4block` fixture remains open because it has neither a terminal solve nor a verified cumulative production gain.
- The exact creator or owner of the root-false `-89` clause payload is not yet classified. Changing Factor without a common-input mismatch would risk fitting a downstream symptom.
- The 120-second queue diagnostic is strong reach evidence, not a proof that every retirement path is covered. The broader N1 retired-Decide item in `PROGRESS.md` remains open.
- The scripted-default demo was not part of the clean aggregate local run because it is known to run slowly in this environment. All 68 other registered tests passed.

4 Next concrete actions

1. From job `1607094`, compare the conflict reasons and learned clauses at conflicts 1007–1010, then trace the first unequal source clause backward to its startup creation or simplification event.
2. Reopen Factor only if the two solvers share the same input grid and differ in exact source reference, quotient emission, update order, or effort stop. Otherwise repair and test the owning simplification/watch lifecycle.
3. Re-run the shipped trace-off production gate, proof/model verification, and earlier P4A fixtures before accepting any change. Require a cumulative production gain; do not enable global conflict reuse from this fixture alone.
4. Once `mod4block` closes, return to `mp1` and implement `SP-VIVIFY-DEDUCE-1` only as the complete Vivify deduction terminal-state machine described in the implementation ledger.

There are no in-flight Slurm jobs owned by this checkpoint.